MONAD ARENA

Beginner-Friendly Project Documentation

A 2D real-time multiplayer browser game where players explore a pixel-art arena, collect crystals, compete with friends, and optionally earn achievements recorded on the Monad blockchain.

Project detail	Plan
Game type	2D top-down multiplayer arena
Target devices	Desktop and mobile browsers
Blockchain	Monad Testnet for development
Game engine	Phaser + TypeScript
Multiplayer	Colyseus server
Web UI	React
Smart contracts	Solidity, added after core gameplay works

Version 1.0 • Planning document • Written in simple language

Contents

- 1. Project in one minute
- 2. The problem and our solution
- 3. How the game works
- 4. Features: first version vs later versions
- 5. Screens and user journey
- 6. Technology explained simply
- 7. How multiplayer works
- 8. How Monad fits into the game
- 9. Security and fair play
- 10. System architecture
- 11. Folder structure
- 12. Data and server messages
- 13. Development roadmap
- 14. Testing plan
- 15. Deployment plan
- 16. Risks and ways to reduce them
- 17. Suggested team roles
- 18. Demo plan and success measures
- 19. Glossary
- 20. Official resources

1. Project in one minute

Project name: Monad Arena

One-line summary: Monad Arena is a real-time 2D multiplayer game where players compete in a shared pixel-art arena, collect crystals, and optionally claim achievements that can be verified on Monad.

Think of a simple top-down arcade game: several players enter the same map, move around, collect glowing crystals, avoid hazards, and try to finish with the highest score. The game should be fun even if a player never connects a crypto wallet.

The first version should focus on the game itself. Blockchain features come after movement, rooms, scoring, and match results work reliably.

Main goal: build a small but polished playable demo, not a huge game with too many features.

2. The problem and our solution

The problem

Some blockchain games focus too much on tokens and transactions and not enough on fun gameplay. In many regular games, achievements and tournament records are stored only by the game company.

Our solution

Monad Arena combines normal real-time gameplay with optional blockchain features. Players can compete first, then connect a wallet if they want to own a badge or claim an eligible achievement.

- **Fast gameplay:** movement and collecting items happen on the game server, not through blockchain transactions.
- **Fairer scoring:** the server checks scores instead of trusting numbers sent by a player's browser.
- **Optional wallet:** players can try the game without a wallet.
- **Verifiable ownership:** a badge recorded by a smart contract can be checked on-chain.

3. How the game works

A normal match

- Open the website and choose a nickname.
- Join a public room or create a private room and share its code.
- When enough players join, the match starts.
- Move around the map and collect crystals.
- Avoid hazards and use a power-up such as a short dash.
- When the timer ends, the server calculates the final score.
- See the results and leaderboard.
- If eligible, connect a wallet and claim a digital achievement.

Starter controls

Action	Desktop	Mobile
Move	WASD or arrow keys	Virtual joystick
Special ability	Spacebar	Action button
Pause/settings	Escape or menu	Settings button

Suggested starting rules

Rule	Initial setting
Players per room	2–8
Match length	3 minutes
Standard crystal	10 points
Rare crystal	25 points
Game view	Top-down 2D map
Modes	Free-for-all, private room, time attack

These are starting design choices. Test them with real players and adjust them based on fun, fairness, and performance.

4. Features: first version vs later versions

MVP: build these first

- A playable map with a character that moves smoothly.
- Multiplayer rooms with 2–8 players.
- Other players visible in real time.
- Crystal spawning, collection, and score updates.
- A timer and final leaderboard.
- Public rooms and private room codes.
- A basic mobile control layout.
- A simple profile and match result screen.
- Wallet connection and one simple achievement badge, after gameplay works.

Later versions

- Team battles and ranked tournaments.
- Seasonal maps, quests, skins, and cosmetic collectibles.
- Tournament rewards with stronger verification.
- Friends list, chat moderation, spectating, and replay tools.
- More maps, characters, and abilities.
- Multiple game servers and a larger-scale matchmaking system.

Do not start with tokens, paid entry fees, a marketplace, or complicated reward pools. Those features add security, legal, and economic work before the core game is proven.

5. Screens and user journey

Screen	What it needs
Landing page	Short explanation, Play button, game preview
Main menu	Nickname, play, settings, optional wallet button
Lobby	Room code, player list, ready status, connection state
Game	Map, player characters, score, timer, crystal count
Results	Final scores, placement, replay button, eligible achievements
Profile	Nickname, stats, match history, badges
Wallet panel	Connected address, network, transaction status

Keep the interface clear. A player should understand how to start a match within a few seconds. On mobile, buttons and text need to be large enough to tap and read.

6. Technology explained simply

Technology	Simple explanation	Why we need it
React	Builds the menus and website screens.	Landing page, lobby, profile, settings
Phaser	Draws and runs the 2D game.	Map, movement, collisions, animations
TypeScript	JavaScript with extra type checks.	Catch mistakes earlier
Colyseus	Helps multiple players share a game room.	Real-time multiplayer and synced state
Node.js	Runs JavaScript on the server.	Hosts game-room logic
Solidity	Language for EVM smart contracts.	Achievements and optional reward claims
Viem	Tools to talk to EVM blockchains.	Read contract data and prepare transactions
Wagmi	React tools for wallet connections.	Connect wallet and show transaction state
PostgreSQL	Stores structured information.	Profiles and match history
Vitest / Playwright	Tools for automated tests.	Test logic and browser journeys

7. How multiplayer works

A multiplayer game needs one trusted place that decides what really happened. In this project, that place is the game server.

Example: collecting a crystal

- The player presses a movement key. The browser sends the intended movement to the server.
- The server checks whether the movement is allowed and updates the player's position.
- The server checks whether the player touched a crystal.
- If the collection is valid, the server adds points and removes the crystal.
- The server sends the new state to everyone in the room.

- Each browser draws the updated game.

The browser should not be allowed to simply send “my score is 5000” and have the server accept it. The server calculates the score itself.

Useful first targets

Item	Initial target
Simulation rate	30–60 updates per second, then performance-test
Network state updates	Start around 15–20 per second and tune
Reconnect grace period	A short period before removing a disconnected player
Match finalization	Server decides the final score and marks it complete

These are tuning goals, not performance guarantees. Test on different networks and devices.

8. How Monad fits into the game

What should be on-chain?

Game feature	Where it should happen	Reason
Movement and collisions	Game server	Must feel immediate
Crystal collection and live score	Game server	Changes constantly
Match history	Database; selected records may go on-chain	Cheaper and simpler
Achievement badge	Monad smart contract	Wallet can prove ownership
Tournament prize claim	Smart contract, in a later version	Can enforce claim rules
Cosmetic item ownership	Optional smart contract	Portable ownership

Monad Testnet setup

Use Monad Testnet while developing. The commonly published testnet details are:

Setting	Value
Network	Monad Testnet
Chain ID	10143
Currency symbol	MON
RPC URL	https://testnet-rpc.monad.xyz

Check the official Monad documentation before deployment in case network details change. Use testnet tokens for development and never put a real private key in frontend code.

Simple badge design

- A player finishes a match that meets the achievement rules.
- The server validates the match and creates a unique result.
- For a simple MVP, the server can sign an achievement claim.

- The player connects a wallet and submits the claim.
- The smart contract checks the authorized signature and whether the claim was already used.
- The contract records the badge and emits an event.

This design trusts the server that signs the result. It does not prove the game was perfectly fair. If the server is dishonest or its signing key is stolen, false claims could be signed. More advanced verification can be added later.

Avoid complex token economics at the start

Do not introduce a tradable token or real-money prize pool in the first version. First prove that people enjoy the game. Token design, reward funding, legal requirements, and smart-contract audits need separate planning.

9. Security and fair play

- The server decides movement limits, crystal collection, cooldowns, and scores.
- Validate all incoming messages and reject unexpected values.
- Limit message rates to reduce spam and simple cheating.
- Use unique match IDs and prevent the same result from being claimed twice.
- Sign reward claims on the server only; never expose the signing key to the browser.
- Bind signatures to the correct chain, contract, game, and expiry.
- Keep admin functions limited and logged.
- Do not store wallet seed phrases or private keys.
- Add chat moderation and a report mechanism if chat is introduced.
- Get a security review before enabling monetary rewards.

A server-authorized signature is a practical MVP approach, but it creates a trust point. Document this honestly in the project description.

10. System architecture

The browser handles display and controls. The multiplayer server runs the match. The database saves persistent records. Monad stores selected ownership and claim records.

```
PLAYER BROWSER
  React UI + Phaser 2D game
          |
          | WebSocket messages
          v
MULTIPLAYER SERVER
  Colyseus rooms + game rules
          |
          +------> DATABASE
          |        profiles, match history
          |
          v
RESULT VALIDATOR
  final score + unique match ID
          |
          v
CLAIM SERVICE
  verifies eligibility and signs claim
          |
          v
MONAD SMART CONTRACT
  badge ownership / optional reward claim
```

The database and blockchain do different jobs. The database is useful for fast app queries and history. The blockchain is useful when a record or asset should be publicly verifiable.

11. Folder structure

```
monad-arena/
  apps/
    web/
      public/assets/
      src/
        components/
        pages/
        game/
          scenes/
          entities/
          systems/
          input/
        multiplayer/
        wallet/
    server/
      src/
        rooms/
        state/
        systems/
        services/
        validation/
  packages/
    shared/
      types/
      constants/
      schemas/
    contracts/
      src/
      test/
      scripts/
  docs/
  .env.example
  package.json
  pnpm-workspace.yaml
  README.md
```

Keep shared types and constants in one place so the browser and server agree about message names, score rules, and game settings.

12. Data and server messages

Main records

Record	Example fields
Player	id, nickname, walletAddress, createdAt
GameRoom	roomId, mode, status, playerCount
Match	matchId, roomId, startedAt, endedAt, status
MatchPlayer	matchId, playerId, score, placement
Achievement	achievementId, playerId, type, earnedAt
Claim	matchId, walletAddress, transactionHash, status

Message names

Message	Purpose
move	Player asks to move in a direction
action	Player asks to use a dash or ability
collect	Player requests an item interaction; server verifies it
ready	Player is ready to start
chat	Text message in a room, if chat is enabled
match:ended	Server reports a finalized match result

Never trust a score or placement sent by a browser. The server must calculate and validate those values.

13. Development roadmap

Phase	Build	Done when
1. Setup	Repository, TypeScript, React, Phaser, basic page	Game loads locally
2. Single player	Map, movement, collision, crystals, score	One person can finish a full match
3. Multiplayer	Colyseus rooms and synced players	Two browsers see each other moving
4. Match rules	Timer, scoring, results, reconnect handling	Server finalizes results correctly
5. UI polish	Lobby, room codes, mobile controls, sound	New player can join and understand the game
6. Testnet badge	Wallet connect and simple achievement contract	Eligible claim works on testnet
7. Deployment	Host web app and persistent game server	Friends can play from separate devices
8. Demo polish	Fix bugs, rehearse demo, document limits	Reliable short live demo

Suggested order: finish phases 1–4 before spending time on tokens, tournaments, or NFT marketplaces. A small working game is better than many unfinished features.

14. Testing plan

Gameplay tests

- Player cannot move through walls or outside the map.
- A crystal can be collected only once.
- Two players cannot both collect the same crystal for duplicate points.
- Score is calculated by the server.
- Match timer ends the match at the expected time.
- Ties and player disconnects are handled consistently.

Multiplayer tests

- Two or more browser sessions can join the same room.
- All players see consistent positions and scores.

- Invalid messages are rejected.
- A disconnected player can reconnect during the grace period.
- Room capacity and room cleanup work correctly.

Blockchain tests

- A valid achievement claim succeeds.
- An invalid signature is rejected.
- An expired claim is rejected.
- A claim cannot be replayed.
- A wallet on the wrong network receives a clear error.
- A failed transaction does not show a false success message.

Browser tests

Use unit tests for scoring and game rules, integration tests for room logic, and browser automation for joining a room and finishing a match. Test at least two separate browsers or devices for multiplayer.

15. Deployment plan

- Put the web frontend on a static web host such as Vercel.
- Deploy the Node.js/Colyseus server to a host that supports persistent WebSocket connections.
- Use a managed PostgreSQL database for persistent records.
- Set environment variables on the server host; do not commit secrets.
- Configure HTTPS/WSS, allowed origins, logs, health checks, and backups.
- Deploy smart contracts to Monad Testnet first.
- Run the complete game from a separate device before sharing the demo link.

A frontend hosting service alone is not enough for a persistent multiplayer server. Check the selected server host's WebSocket, process lifetime, region, and scaling support.

16. Risks and ways to reduce them

Risk	Why it matters	Reduction plan
Cheating	Browser values can be changed	Server-authoritative rules and validation
Lag	Players may see different states	Tune update rates, interpolation, test regions
Scope creep	Too many features delay launch	Finish the MVP before adding modes
Server failure	Rooms may stop unexpectedly	Health checks, logs, reconnects
Bad contract claim	Wrong user could receive a badge	Signature checks, replay protection, tests
Signer compromise	Fraudulent results could be signed	Secure key storage, limits, rotation plan
Wallet friction	Some players do not use crypto	Allow play without a wallet
Unbalanced gameplay	One strategy dominates	Playtests and tune scoring/power-ups

17. Suggested team roles

Role	Main responsibility
Game developer	Phaser map, controls, animation, game feel
Backend developer	Colyseus rooms, scoring, validation, database
Smart contract developer	Badge contract, tests, testnet deployment
UI / frontend developer	React screens, responsive layout, wallet UI
Product / QA	Rules, playtesting, bug tracking, demo script

One person can cover multiple roles for a small hackathon. If the team is small, prioritize gameplay and multiplayer before blockchain features.

18. Demo plan and success measures

A 2–3 minute demo

- Open the game and show the main menu.
- Join the same room from two browser windows or devices.
- Move both players around and collect crystals.
- Show the shared score and timer.
- Finish a match and show the final server-calculated results.
- Connect a testnet wallet and claim a badge.
- Open the transaction or contract event to show the on-chain record.

How to measure success

Measure	Initial target
Playable match	A complete match from start to results
Multiplayer	At least 2 clients in one room
Scoring	No duplicate crystal scoring in tests
Reconnect	Short disconnect does not instantly lose the session
Wallet	Testnet claim succeeds and duplicate claim fails
Usability	A new tester can start without detailed help

These are MVP acceptance targets, not claims about completed functionality.

19. Glossary

Term	Simple meaning
2D	A game world shown with height and width, rather than full 3D.
Multiplayer room	A shared game session where players meet.
Authoritative server	The server that decides the official game state.
WebSocket	A connection that lets the browser and server exchange messages quickly.
Smart contract	A program deployed on a blockchain.
Wallet	An app used to control blockchain accounts and approve transactions.
Testnet	A blockchain network for testing with non-production tokens.
NFT / digital badge	A blockchain token or record that can represent an owned achievement.
RPC	A service endpoint apps use to communicate with a blockchain.
Replay protection	A check that prevents an old valid request from being used again.
MVP	Minimum Viable Product: the smallest useful version to test the idea.

20. Official resources

Use the official documentation for exact setup commands and version-specific instructions.

- Monad documentation — network, development, and deployment details.
- Phaser documentation — browser 2D game engine.
- Colyseus documentation — multiplayer rooms and synchronization.
- Viem documentation — EVM blockchain client.
- Wagmi documentation — wallet integration for React.
- OpenZeppelin Contracts — reusable smart contract components.
- Playwright documentation — browser testing.

Network details and software APIs can change. Verify the current official documentation before implementation or deployment.

Final recommendation: build the game loop first, make multiplayer stable second, and add Monad achievements third. Keep the first release small, playable, and easy to demonstrate.